

CSC 4520/6520

Design & Analysis of Algorithms

CHAPTER 1: INTRODUCTION

ABDULLAH BAL, PHD

Objectives



Data Structures



Linear Data
Structures



Graphs



Trees

Data Structures

- Since the vast majority of algorithms of interest operate on data, particular ways of organizing data **play a critical role in the design and analysis of algorithms.**
- A ***data structure*** can be defined as a particular scheme of organizing related data items.

Data Structures

- Computers may be fast,
- But they are not infinitely fast.
- Computing time is therefore a bounded resource, which makes it precious.
- Memory may be inexpensive,
- But it is neither infinite nor free.
- That's why, we should choose algorithms that use the resources of time and space efficiently.

Data Structures

- A data structure is a way to **store and organize** data in order to facilitate **access and modifications**.
- Using the appropriate data structure or structures is an important part of **algorithm design**.
- **No single data structure** works well for all purposes, and so **we should know the strengths and limitations of several of them**.

Fundamental Data Structures

- List
 - Array
 - Linked List
 - String
- Stack
- Queue
- Priority Queue
- Graph
- Tree
- Set and Dictionary

Linear Data Structures: Array - Linked List

- The two most important elementary data structures are the **array** and the **linked list**.
 - A (one-dimensional) **array** is a sequence of n items of the same data type that are stored contiguously in computer memory and made accessible by specifying a value of the array's **index**
 - A **linked list** is a sequence of zero or more elements called **nodes**, each containing two kinds of information: some data and one or more links called **pointers** to other nodes of the linked list.

Linear Data Structures: List

- A **list** is a finite sequence of data items, i.e., a collection of data items arranged in a certain linear order.
- The basic operations performed on this data structure are **searching** for, **inserting**, and **deleting** an element.
- Two special types of lists, stacks and queues, are particularly important.

Linear Data Structures: Stack

- A **stack** is a list in which insertions and deletions can be done **only at the end**.
- This end is called the **top** because a stack is usually visualized not horizontally but vertically—akin to a stack of plates whose “operations” it mimics very closely. As a result, when elements are added to (pushed onto) a stack and deleted from (popped off) it, the structure operates in a **“last-in–first-out” (LIFO)** fashion— exactly like a stack of plates if we can add or remove a plate only from the top.

Linear Data Structures: Queue

- A **queue**, on the other hand, is a list from which elements are deleted from one end of the structure, called the **front** (this operation is called **dequeue**), and new elements are added to the other end, called the **rear** (this operation is called **enqueue**).
- Consequently, a queue operates in a “first-in–first-out” (FIFO) fashion—akin to a queue of customers served by a single teller in a bank.

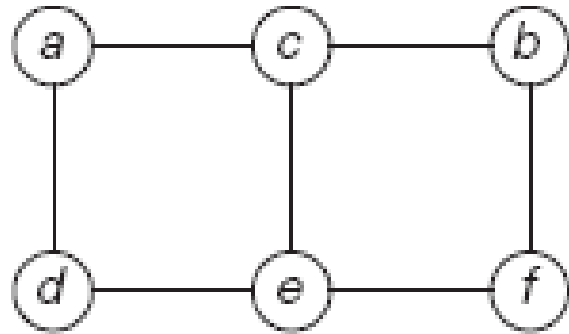
Linear Data Structures: Priority Queue - Heap

- Many important applications require selection of an item of the highest priority among a dynamically changing set of candidates.
- A data structure that seeks to satisfy the needs of such applications is called a priority queue. A **priority queue** is a collection of data items from a totally ordered universe (most often, integer or real numbers).
- The principal operations on a priority queue are finding its **largest element**, deleting its largest element, and adding a new element.
- A better implementation of a priority queue is based on an ingenious data structure called the **heap**.

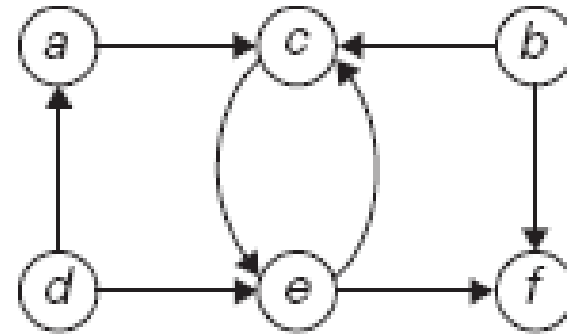
Graphs

- A graph is informally thought of as a collection of points in the plane called “**vertices**” or “**nodes**,” some of them connected by line segments called “**edges**” or “**arcs**.”
- Formally, a **graph** $G = \langle V, E \rangle$ is defined by a pair of two sets: a finite nonempty set V of items called **vertices** and a set E of pairs of these items called **edges**.
- If these pairs of vertices are unordered, i.e., a pair of vertices (u, v) is the same as the pair (v, u) , we say that the vertices u and v are **adjacent** to each other and that they are connected by the **undirected edge** (u, v) .
- If a pair of vertices (u, v) is not the same as the pair (v, u) , we say that the edge (u, v) is **directed** from the vertex u , called the edge’s **tail**, to the vertex v , called the edge’s **head**.
- A graph whose every edge is directed is called **directed**. Directed graphs are also called **digraphs**.

Graph Representations



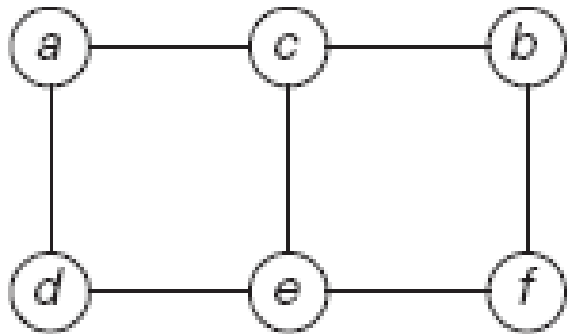
Undirected graph



Directed graph

Graph Representations

- Graphs for computer algorithms are usually represented in one of two ways:
 - The adjacency matrix
 - The adjacency lists.



	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
<i>a</i>	0	0	1	1	0	0
<i>b</i>	0	0	1	0	0	1
<i>c</i>	1	1	0	0	1	0
<i>d</i>	1	0	0	0	1	0
<i>e</i>	0	0	1	1	0	1
<i>f</i>	0	1	0	0	1	0

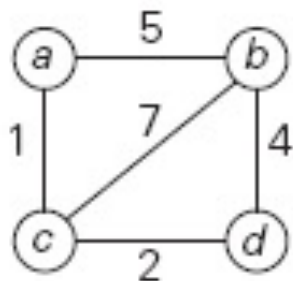
Adjacency matrix

<i>a</i>	→	<i>c</i>	→	<i>d</i>	
<i>b</i>	→	<i>c</i>	→	<i>f</i>	
<i>c</i>	→	<i>a</i>	→	<i>b</i>	→ <i>e</i>
<i>d</i>	→	<i>a</i>	→	<i>e</i>	
<i>e</i>	→	<i>c</i>	→	<i>d</i>	→ <i>f</i>
<i>f</i>	→	<i>b</i>	→	<i>e</i>	

Adjacency lists

Graphs: Weighted Graphs

- A **weighted graph** (or weighted digraph) is a graph (or di-graph) with numbers assigned to its edges.
- These numbers are called **weights** or **costs**.
- An interest in such graphs is motivated by numerous real-world applications, such as finding the **shortest path** between two points in a **transportation** or **communication** network or the **traveling salesman problem**.



Weighted graph.

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>
<i>a</i>	∞	5	1	∞
<i>b</i>	5	∞	7	4
<i>c</i>	1	7	∞	2
<i>d</i>	∞	4	2	∞

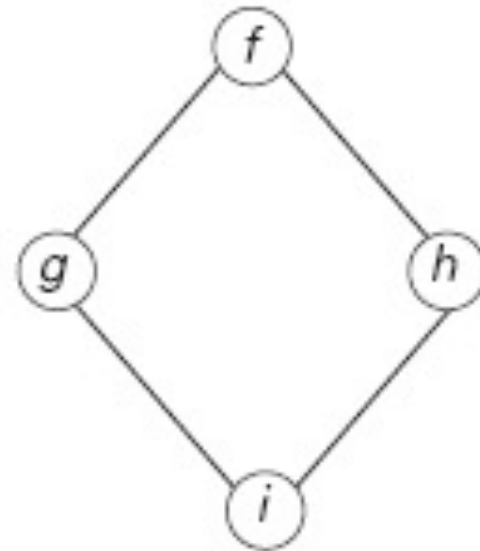
Its weight matrix

<i>a</i>	$\rightarrow b, 5 \rightarrow c, 1$
<i>b</i>	$\rightarrow a, 5 \rightarrow c, 7 \rightarrow d, 4$
<i>c</i>	$\rightarrow a, 1 \rightarrow b, 7 \rightarrow d, 2$
<i>d</i>	$\rightarrow b, 4 \rightarrow c, 2$

Its adjacency lists.

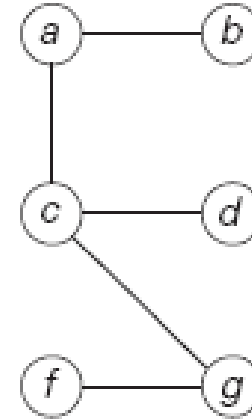
Graphs: Cycle - Acyclic

- A **cycle** is a path of a positive length that starts and ends at the same vertex and does not traverse the same edge more than once.
- For example, f, h, i, g, f is a cycle in the following graph.
- A graph with no cycles is said to be **acyclic**.

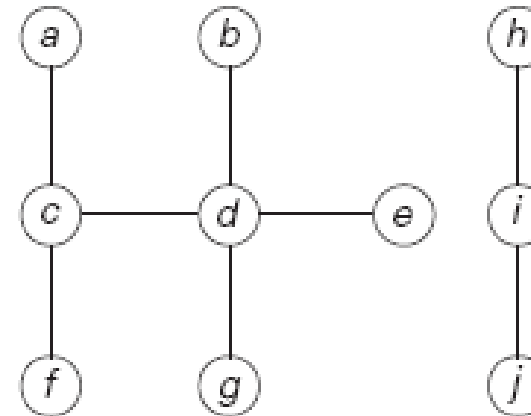


Trees

- A **tree** is a connected acyclic graph.

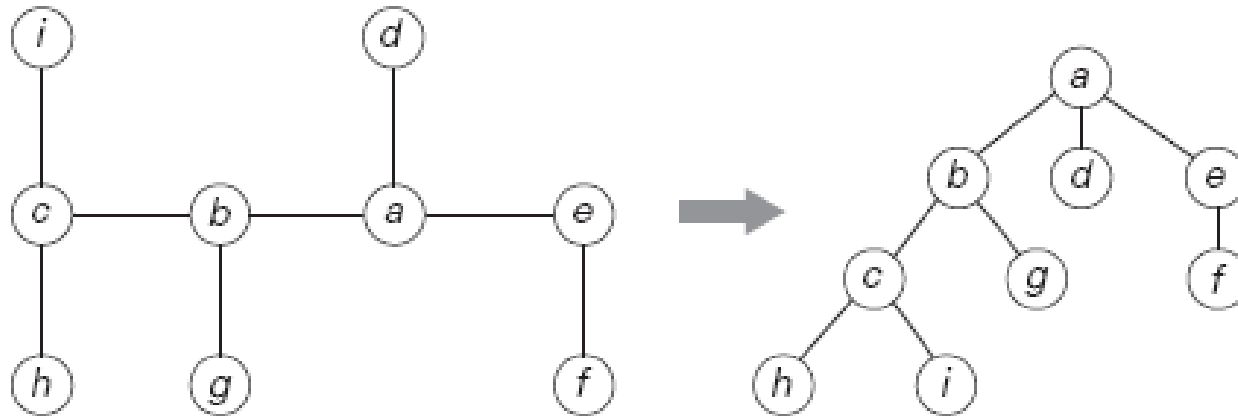


- A graph that has no cycles but is not necessarily connected is called a **forest**: each of its connected components is a tree.



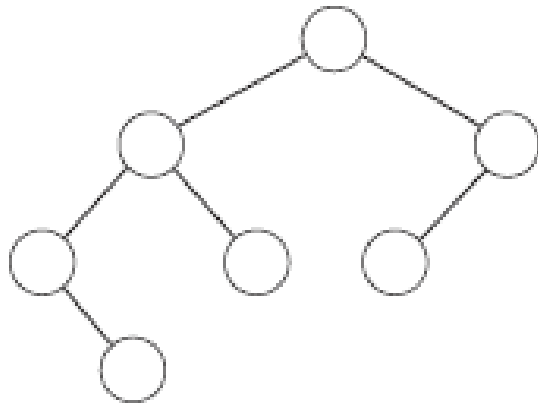
Trees: Rooted Trees

- A **rooted tree** is usually depicted by placing its root on the top (level 0 of the tree), the vertices adjacent to the root below it (level 1), the vertices two edges apart from the root still below (level 2), and so on.
- A transformation from a free tree to a rooted tree.

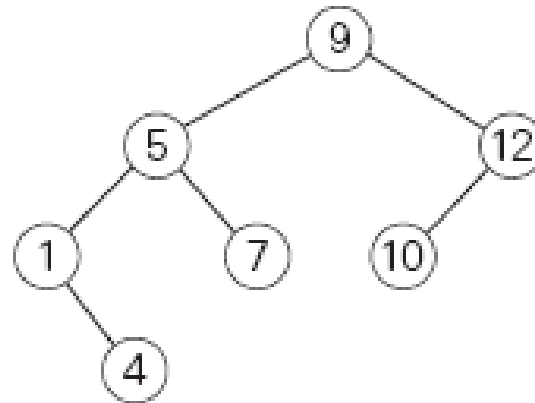


Trees: Ordered Trees – Binary Trees

- An **ordered tree** is a rooted tree in which all the children of each vertex are ordered.
- It is convenient to assume that in a tree's diagram, all the children are **ordered left to right**.
- A **binary search tree** can be defined as an ordered tree in which every vertex has no more than two children and each child is designated as either a **left child** or a **right child** of its parent; a binary tree may also be empty.



Binary tree.



Binary search tree.

Summary

- Data Structures
- Linear Data Structures
- Graphs
- Trees

Supportive Materials

- *Introduction to The Design and Analysis of Algorithms (3rd Edition)* by Anany Levitin, Pearson.
- *Introduction to Algorithms*, by T.Cormen, C. Leiserson, R.Rivest and C.Stein, MIT Press, 3rd Edition.